

Министерство цифрового развития, связи и массовых коммуникаций РФ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Сибирский государственный университет
телекоммуникаций и информатики» (СибГУТИ)

Кафедра телекоммуникационных систем и вычислительных средств

Практическое задание № 5

Циклический избыточный код. CRC

Выполнил: студент 3 курса

гр. ИА-331

Помелова А.В.

Новосибирск 2025

ЦЕЛЬ ЗАНЯТИЯ

Получить представление о том, как осуществляется проверка на наличие ошибок в пакетах с данными в современных системах связи (Error detection) посредством использования циклического избыточного кода CRC (Cyclic Redundancy Check).

КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

CRC (Cyclic Redundancy Check) - циклический избыточный код, используемый в современных системах связи для обнаружения ошибок в пакете данных.

CRC - набор дополнительных битов (контрольная сумма), вычисляемых по заранее известному алгоритму и передаваемых вместе с данными.

Назначение CRC — обнаружение ошибок при передаче по каналу связи.

Принцип работы CRC

CRC - остаток от двоичного деления исходной битовой последовательности на специальное двоичное число - порождающий полином.

- если степень полинома = n , то длина CRC = $n-1$ бит
- деление выполняется с помощью операции XOR

Перед делением в конец данных добавляют $n-1$ нулей.

Порождающий полином (делитель)

Делитель записывается как двоичное число, где каждый бит — коэффициент полинома:

$$G(x) = x^{n-1} + x^{n-2} + \dots + x^2 + x + 1$$

Полином полностью определяет, какие ошибки способен обнаружить CRC.

Вычисление CRC на передающей стороне

Шаги:

1. взять исходные данные
2. добавить справа $n-1$ нулей
3. выполнить «деление» с помощью XOR
4. остаток записать как CRC

CRC добавляется в конец данных: [данные | CRC]. Эта битовая последовательность и отправляется по каналу связи.

Проверка CRC на приёмной стороне

Полученный пакет снова делится на тот же порождающий полином.

- если остаток = 0 - ошибка не обнаружена - пакет принят
- если остаток $\neq 0$ - обнаружена ошибка

Основные свойства CRC

CRC гарантированно обнаруживает:

- все одиночные ошибки
- большинство ошибок длиной $\leq$ степени полинома
- все ошибки с нечётным числом инверсий

Поэтому CRC широко используется в мобильных сетях.

ЗАДАНИЕ ДЛЯ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

В рамках данной работы студенты должны научиться вычислять CRC-последовательности, а также на их основании детектировать ошибки.

- 1) Напишите программу на языке C/C++ для вычисления CRC для пакета данных длиной N бит ($N = 20 + \text{порядковый номер в журнале}$) и определения факта наличия ошибки при передаче пакета по каналу связи.
- 2) Порождающий полином G для делителя выберите в соответствии с вариантом. Номер варианта – порядковый номер в журнале группы.

$$G = x^7 + x^6 + x + 1$$

19	$G = x^7 + x^6 + x + 1$
----	-------------------------

- 3) Добавьте полученный остаток от деления на G к пакету исходными данными и на приемной стороне вычислите повторно остаток от деления пакета с данными+CRC на полином G . Определите есть ли ошибка в

принятом пакете. Выведите в окно терминала полученное значение CRC и отчет об ошибках в принятом пакете.

- 4) Возьмите N, равное 250 битам. Прodelайте п.1-3
- 5) Сделайте цикл из $250 + \text{CRC length}$ итераций и в этом цикле по очереди искажайте по одному биту – с 0-го до $250 + \text{CRC} - 1$, проверьте в соответствии с п.3 обнаружена ли ошибка на приемной стороне и выполните подсчет того сколько раз за этот цикл приемник обнаружил и не обнаружил ошибки. Результат выведите в окно терминала.
- 6) Составьте отчет. Отчет должен содержать титульный лист, содержание, цель и задачи работы, теоретические сведения, исходные данные, этапы выполнения работы, сопровождаемые скриншотами и графиками, демонстрирующими успешность выполнения, и промежуточными выводами, результирующими таблицами, ответы на контрольные вопросы, и заключение и **ссылка в виде QR-кода на репозиторий с кодом (git)**.

ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

$N = 39$ - длина пакета данных в битах.

$\text{CRC_len} = 7$ — длина контрольной суммы (остатка), потому что полином степени 7

Полином $G = x^7 + x^6 + x + 1$ --- 1 1 0 0 0 0 1 1 --- массив {1,1,0,0,0,0,1,1}

Создаётся случайный пакет из 39 бит: каждый бит - 0 или 1.

Это исходное сообщение, которое отправится по каналу связи.

```
for (int i = 0; i < N; i++){  
    data[i] = rand() % 2;  
}
```

Вычисление CRC (на передающей стороне)

- 1) Копируем данные в буфер и добавляет 7 нулей в конец
- 2) Делим (с помощью XOR) на полином G
- 3) Сохраняем остаток - это и есть CRC (7 бит).

```
void crc(int input[], int len, int rem[]) {  
    int tmp[len + CRC_len];
```

```

for (int i = 0; i < len; i++) tmp[i] = input[i];
for (int i = len; i < len + CRC_len; i++) tmp[i] = 0;

for (int i = 0; i < len; i++) {
    if (tmp[i] == 1) {
        for (int j = 0; j <= CRC_len; j++) {
            tmp[i + j] ^= polynom[j];
        }
    }
}

for (int i = 0; i < CRC_len; i++)
    rem[i] = tmp[len + i];
}

```

Формирование кадра

```

for (int i = 0; i < N; i++) frame[i] = data[i];
for (int i = 0; i < CRC_len; i++) frame[N + i] = crc[i];

```

- К исходным данным прикрепляем CRC в конце - frame = [data | crc]
- Получается полный кадр длиной $39 + 7 = 46$ бит

Полученный кадр снова делится на полином G:

- если остаток совпадает с CRC, то ошибок нет
- если отличается - ошибка обнаружена

```

int is_valid(int frame[], int len) {
    int rem[CRC_len];
    calc_crc(frame, len, rem);    // считаем CRC от первых len бит данных

    for (int i = 0; i < CRC_len; i++) {
        if (rem[i] != frame[len + i]) {
            return 0;    // CRC не совпадает - ошибка
        }
    }
    return 1;    // CRC совпадают - ошибок нет
}

```

```

printf(is_valid(frame, N) ? "Нет ошибок\n" : "Есть ошибки\n");

```

Проверка способности CRC обнаруживать одиночные ошибки

Выполняется цикл:

- по очереди инвертируется каждый бит кадра
- проверяется, обнаружена ли ошибка

- затем бит возвращается обратно

Подсчитывается:

- detected - сколько ошибок CRC обнаружила
- missed - сколько ошибок CRC не обнаружила

```
int detected = 0, missed = 0;
for (int i = 0; i < length; i++) {
    frame[i] ^= 1;
    if (is_valid(frame, N)) missed++; else detected++;
    frame[i] ^= 1;
}

printf("Проверено: %d Пропущено: %d\n\n", detected, missed);
```

```
=== N = 39 ===
CRC: 1011110
Нет ошибок
Проверено: 46 Пропущено: 0
```

"Проверено: 46" - корректно перебрал все 46 возможных одиночных ошибок
"Пропущено: 0" - CRC обнаруживает 100% одиночных ошибок

Увеличение N = 250

Далее повторяются:

- генерация данных
- вычисление CRC
- формирование кадра
- проверка одиночных ошибок

```
=== N = 250 ===
CRC: 1010000
Нет ошибок
Проверено: 257 Пропущено: 0
```

"Проверено: 257" - проверил каждый бит
"Пропущено: 0" - CRC обнаружила все одиночные ошибки

ВЫВОДЫ

В ходе выполнения я получила представление о том, как осуществляется проверка на наличие ошибок в пакетах с данными в современных системах связи (Error detection) посредством использования циклического избыточного кода CRC (Cyclic Redundancy Check).

КОД

Ссылка на github



КОНТРОЛЬНЫЕ ВОПРОСЫ

1) Для чего в мобильных сетях используются CRC-проверки?

CRC используется для обнаружения ошибок, возникающих при передаче данных по радиоканалу. Из-за шума, замираний, интерференции и помех отдельные биты могут искажаться, поэтому CRC позволяет:

- проверить целостность принятого пакета,
- определить, были ли ошибки при передаче,
- отбраковать неверные пакеты и инициировать повторную передачу.

2) Что такое порождающий полином?

Порождающий полином - специально выбранный двоичный полином, который используется как делитель при вычислении CRC.

Он определяет:

- длину CRC ($n-1$ бит, где n — степень полинома + 1),
- набор ошибок, которые CRC гарантированно обнаружит,
- свойства и устойчивость к burst-ошибкам.

3) Как вычислить CRC для пакета с данными?

- Взять исходный пакет данных.
- Добавить $n-1$ нулей в конец (где n — длина порождающего полинома).
- Выполнить побитовое деление (операцией XOR) расширенного пакета на порождающий полином.
- Полученный остаток от деления — это CRC (контрольная сумма).
- Добавить CRC в конец к исходным данным